

Projet de programmation Licence 2^{ème} année

Parabox-like game

Thierry Hamon

Décembre 2025

Principe Le projet de programmation consiste à implémenter votre propre version de Patrick's Parabox ¹. Il s'agit d'un Sokoban ² (vieux jeu-vidéo de puzzle où l'on doit pousser des boîtes au bon endroit) remis au goût du jour avec plein de mécanismes très originaux, notamment l'idée de pouvoir entrer dans les boîtes et d'avoir des boîtes récursives.

Langage Le projet doit être réalisé en *Java*. Pour la partie graphique, vous pouvez utiliser une bibliothèque graphique : *Swing* ou *JavaFX*. Un CM sera donné sur le premier pour briser la glace, mais le but est d'apprendre par vous-même (il y a aussi un CM sur *git* dans le même esprit).

Taille des groupes Le projet se focalise sur le travail de groupe et la quantité de travail demandée est importante. Vous devez donc faire des groupes allant de 4 à 8 personnes. ³

Choisir un groupe avec plus de membres sera très valorisé dans la note finale. Évitez d'intégrer un membre unique à un groupe d'amis qui se connaissent bien ; par exemple, si vous êtes un groupe de trois amis, chercher à intégrer 2 membres plutôt que un, cela évite les situations d'isolement.

Un canal *mattermost* sera ajouté pour les annonces de recherche de groupe/membres.

Chaque membre du groupe doit être responsable d'une tâche du projet. Cela ne signifie pas que le responsable ne travaille que sur cet aspect, car les tâches sont inégales en terme de travail à fournir. Chaque personne supplémentaire dans le groupe rajoutera une tâche.

Remerciement Le projet a pu se faire grâce à l'auteur, Patrick Traynor, qui nous autorise à utiliser son concept. Il a aussi fourni des *blue-prints* de niveaux qui vous seront transmis par ailleurs. Veuillez respecter son oeuvre et ne pas mettre votre code/jeu sur internet sans lui demander l'autorisation au préalable

1. <https://www.patricksparabox.com/>

2. <https://fr.wikipedia.org/wiki/Sokoban>

3. Je peux accepter les groupes de 4, mais les exigences seront les mêmes, et avec le même barème.

(en anglais). Et même dans ce cas, n'utilisez que pas les exemples fournis mais des niveaux que vous auriez écrit.

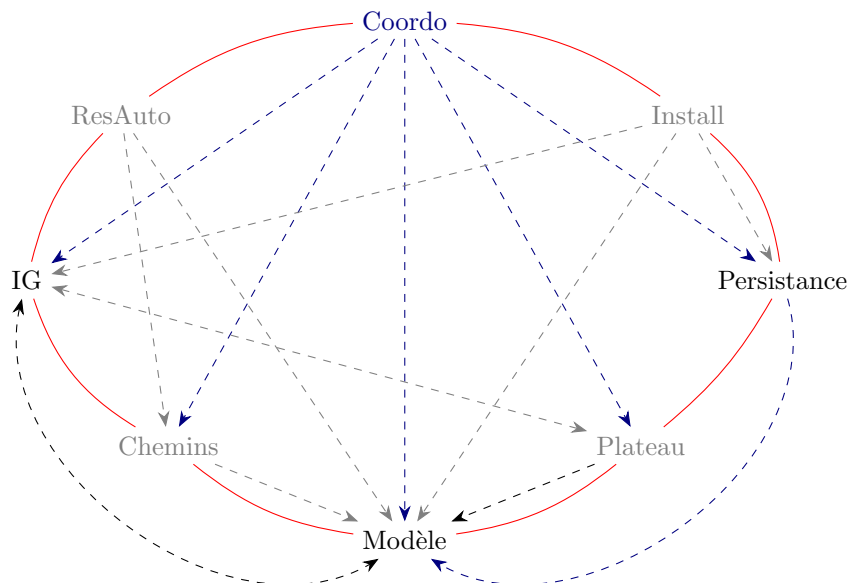
1 Tâches

Les tâches suivantes doivent être effectuées, avec un responsable pour chaque phase. Les 4 premières sont obligatoires pour tous, les quatre suivantes le sont que pour des groupes de taille 5, 6, 7 voire 8 :

- 4+ : Coordinateur ; coordination
- 4+ : Interface Graphique (IG) ; l'interface graphique avec les contrôles demandés,
- 4+ : Modèle (Mod) ; le modèle mémoire autorisant les mouvements possibles uniquement,
- 4+ : Persistance (Pers) ; l'ouverture et l'enregistrement de niveau format imposé, ainsi que la version textuelle,
- 5+ : Installation et Déploiement (Install) ;
- 6+ : Chemins (Chem) ; la résolution automatique de chemin,
- 7+ : Edition de plateau (Plat) ; la création et la modification de plateau.
- 8+ : Résolution automatique (ResAuto) ; un essai de résolution automatique de puzzle,

Ces tâches sont détaillées dans les annexes dédiées. Le responsable de chaque tâche doit connaître son annexe. Mais avant ça, vous devez bien avoir en tête que personne ne doit se cantonner à ses propres tâches : la répartition du travail décrite ci-dessus est à peu près équilibrée pour un travail valant une note de 8-10 (en fonction du rapport). Lorsque vous irez plus loin, certains rôles seront vite désœuvrés tandis que d'autres (rôles 2 et 3 notamment) demanderont plus de travail. Il vous faudra alors travailler ensemble et vous entraider.

Interactions entre les rôles



De plus, certains auront à travailler ensemble dès le départ, assurant ainsi la cohésion du groupe. Le graphe ci-dessus présente les interactions naturelles ; une ligne pleine signifie que vous devez comprendre, voir relire le code de l'autre ; une flèche en pointillé signifie que vous devez connaître et comprendre la structure (classes, attributs et méthodes publiques de l'autre). Notez que les lignes vont dans les deux sens et les pointillés sont directionnels. Le maintien de ces liens sera évalué.

Remarquez que les rôles à gauche sont les rôles *frontend*/affichage contrairement aux rôles de droites qui sont plus *backend*/contenu. De même, les rôle en haut sont plus des rôles de supervision tandis que les rôles du bas sont des rôles techniques (d'où les flèches de haut en bas).

On demande aussi à ce que tout le code produit soit “reviewed”⁴ par un autre membre du groupe. Il s'agit, pour lui, de lire le code et de noter :

- les bugs qu'il voit,
- ce qu'il ne comprend pas,
- ce qu'il aurait fait autrement,
- tout commentaire qui lui semble pertinent.

Tout cela peut se faire de vive voix, mais c'est encore mieux de l'ajouter dans des commentaires de code, qui seront versionnés, lus, puis effacés. De cette manière l'historique de *git* gardera la trace de ceux-ci.

4. La traduction devrait être “relu”, mais on veut ici faire plus qu'une simple relecture, il est attendu un retour critique et constructif du relecteur.

Fonctionnalités attendues ou possibles du jeu

1.1 Graphique et contrôles

La qualité esthétique n'est pas très importante (on n'est pas graphiste), mais sera prise en compte.

Sokoban classique : Il faut que l'on arrive à distinguer notre personnage, les murs, les boîtes, les emplacements où aller ainsi que les boîtes posées sur des cibles.

Contrôles classique : Déplacement si possible avec les flèches directionnelles.

Retour en arrière : Retour à la position précédente (si existant) avec **Ctrl-z**.

Résolution de chemin (5+) : Déplacement automatique si possible en cliquant sur l'écran en utilisant la fonction dédiée (responsabilité de votre camarade).

Sokoban récursif : Il faut afficher le contenu des boîtes à une profondeur de 4, ainsi que le contenu extérieur de la boîte courante sur 1/4 case de chaque côté.

Mondes colorés : Chaque boîte-monde doit avoir une couleur différente.

Indices : En utilisant la résolution automatique de puzzle, la touche "I" doit afficher un indice.

1.2 Mouvements

Il est impératif de pouvoir faire les mouvement autorisés et seulement ceux-ci.

Sokoban classique : Les cibles ne sont pas neutres du point de vue déplacement. On peut se déplacer (en haut/bas/droite/gauche) vers une case vide. On ne peut pas se déplacer vers un mur. On peut se déplacer vers une boîte tant que la boîte peut aussi se déplacer dans la même direction, avec les mêmes règles (elle est alors déplacée aussi).

Sokoban(s) récursif(s) : Dans le cas du sokoban récursif, comme pour le sokoban classique, il faut utiliser du "back-tracking". En effet, on peut sortir/entrer de/dans plusieurs boîtes en un seul mouvement et s'apercevoir à la fin qu'il y a un mur. On doit définir un nouveau type de cellules, car en plus des

boites simples, il y a maintenant des boites *contenant* un autre monde. Pour faciliter la conception, on peut référencer les mondes par des entiers.

Truc de l'auteur : vous pouvez ne pas avoir à implémenter les cas de la boite simple en disant qu'une boite simple est une boîte-monde correspondant à un monde de taille 1 contenant uniquement un mur. On perd alors un peu en efficacité (pas en complexité!), mais on gagne en taille de code, et en temps de correction de bugs. On parle de factorisation de code, car on a ramené le code relatif aux boites simple à une sous-partie du code relatif aux boites mondes. Une autre possibilité plus propre, plus efficace, mais beaucoup plus difficile, est d'implémenter la boite monde comme une sous-classe de la boite simple, de faire des méthodes pour les bouts à factoriser, et de simplement changer l'ordre d'appel.

1.3 Fichier de niveau

Les niveaux sont représentables par des chaînes de caractères ASCII. Cela permet de charger un niveau depuis un fichier texte et de l'enregistrer dans un autre fichier texte. Le responsable de cette tâche doit aussi fournir une version terminal de la partie graphique afin de pouvoir faire des tests au plus vite.

Un niveau est constitué de plusieurs mondes (un seul pour le Sokoban classique). Ils sont écrits à la suite les uns des autres.

Un monde commence par une lettre (son nom) un nombre (sa taille) séparés par un espace avant de passer à la ligne (avec `\n` ou `\r\n`).

Puis un monde de taille n contient n lignes de n caractères qui sont

- **dans le Sokoban classique** :⁵ un espace ' ' pour une case vide, un dièse '#' pour un mur, un '\$' pour une boite, un '@' pour le personnage, un point '.' pour une cible, un '*' pour une boite sur une cible, et un '+' pour un personnage sur une cible.
- **dans le Sokoban récursif** : Une boite peut être associée à n'importe quelle lettre (majuscule si sur une cible), sémantiquement elle contient le monde du même nom si celui-ci existe.

Attention : cette tâche doit être effectuée très vite afin de permettre à chacun de tester son code. Le responsable ne doit donc pas hésiter à débaucher ses camarades au début en leur demandant de l'aider. À l'opposé, cette tâche finie, le responsable pourra aider les autres.

1.4 Enregistrement des parcours ou des solutions

Les parcours ou les solutions doivent être enregistrés en utilisant l'encodage décrit ici : http://www.sokobano.de/wiki/index.php?title=Puzzle_format#Solution

5. voir http://www.sokobano.de/wiki/index.php?title=Puzzle_format

1.5 Résolution automatique de chemin

Lorsque l'utilisateur clique sur une case vide, vous devez lancer un algorithme de parcours (voir Section B de l'annexe) qui cherchera le plus court chemin du personnage au point cliqué sans bouger aucune boîte. Si aucun chemin n'est trouvé alors rien ne se passe, sinon il faudra animer le déplacement, c'est-à-dire voir le personnage se déplacer case par case jusqu'au point d'arrivée.

Dans le Sokoban récursif, la notion de "chemin sans bouger de boîte" n'est pas réversible : si on sort d'une boîte, on ne peut pas forcément y ré-entrer sans la bouger. On utilise donc un algorithme dirigé.

1.6 Résolution automatique de puzzle

Il s'agit maintenant de chercher une solution complète au problème et de l'afficher sous forme d'indices lorsque l'utilisateur appuie sur la touche "I".

Cette partie est beaucoup plus libre et exploratoire, c'est pourquoi elle n'est pas obligatoire : seuls les groupes de 8 doivent avoir fait des essais où une version heuristique suffit.

2 Évaluation

Rendu d'initialisation le 2 février 2026 Vous devrez, le 2 février 2026 :

- avoir trouvé les membres de votre groupe et avoir fait une première réunion,
- avoir créé un canal mattermost privé en invitant les membres de votre groupe et votre enseignant,
- y avoir déposé un "diagramme des classes" prévues, avec des patates nommés pour les classes, leurs attributs et méthodes, ainsi que des flèches pour l'héritage
- y avoir déposé un "diagramme de Gantt" de ce que vous envisagez de faire, avec une ou plusieurs ligne par membres indiquant les tâches à faire et à quel moment vous pensez les faire.

Dans le cas des diagrammes, je ne demande rien de très professionnel, des dessins à la main suffisent. Ils n'auront pas non plus à être respectés, le but est de structurer vos idées. Par contre il faudra expliquer tous les changements apportés dans le rendu final.

Rendu final début avril À la fin du projet, il vous faudra rendre le code produit (toutes leurs versions, qu'elles soient finies ou pas), un *makefile* et/ou POM (Maven) et/ou tout fichier permettant une installation facile, ainsi qu'un *README*. Le code doit être facilement compiler et lancer sur *Linux*. Cette partie sera assurée par le rôle "Installation et Déploiement" s'il est affecté, sinon par le "Coordinateur"

On demande aussi un rapport **commun** au format PDF de 3 pages maximum contenant :

- la répartition des rôles et l’organisation du travail au sein du groupe,
- le diagramme des classes final, en commentant les changements sur l’initial,
- le diagramme de Gantt final, en commentant les changements sur l’initial,
- un listing des bugs encore présents (un bug connu est moins grave qu’un bug ignoré),
- une auto-évaluation des point forts et points faibles de votre travail,
- un listing des difficultés principales rencontrées par le groupe,
- la description d’une solution (même partielle ou pas totalement correcte) utilisée pour l’une des difficultés,

et un rapport **individuel** au format PDF d’une ou deux pages chacun contenant :

- une description de votre contribution (1/2 page maximum),
- une réflexion libre sur ce que vous auriez fait différemment si vous aviez recommencé le projet au moment de la rédaction du rapport,
- une dernière partie libre sur votre retour d’expérience.

Le rapport joint et le rapport individuel sont très importants (peut être plus que le rendu technique). Le rapport individuel doit vous interroger sur votre implication dans le projet.

La qualité de la rédaction sera prise en compte.

Le rendu du projet se fait via un dépôt Git sur lequel seront invités les enseignants et tuteurs et dont le lien sera donné dans le canal Mattermost de votre groupe. Le rapport commun sera également transmis sur le canal Mattermost de votre groupe. Les rapports individuels seront envoyé via Mattermost mais en privé.

Soutenances Vous aurez une soutenance en groupe après les partiels de mi-semester (mi avril / mi mai). Vous ferez une présentation globale du travail de groupe et une présentation individuelle de votre travail personnel. Les présentations doivent se faire avec un support au format PDF (envoyé ou déposé sur Moodle, au préalable). La présentation globale ne devra pas durer plus de 10 min. Les présentations individuelles ne devront pas prendre plus de 5 minutes. Le temps de présentation doit être respecté, sinon je l’interromprai. Chaque présentation sera suivie de 5 minutes de questions/commentaires sur votre travail. C’est la partie importante. Je poserai des questions sur votre avancement, votre implication et vos interactions.

Notation La note comportera trois parties :

- une partie commune dépendante de l’avancée de votre projet, mais aussi des spécificités de votre groupe (taille, problématique inattendue, etc...).
- une partie individuelle sur votre implication et respect des consignes,
- et une partie de “rôle” dépendant de la manière dont votre rôle a été rempli.

Éléments évalués Lors de l’évaluation, en plus des documents ci-dessus, je regarderai l’historique de votre dépôt *Git*, de votre canal privé Mattermost, de

vos posts sur les canaux publics et de documents supplémentaires transmis et acceptés par tous (comptes-rendus de réunions notamment). Éventuellement, je peux accepter un rapport envoyé en privé s'il y a des éléments personnels dedans. En aucun cas, je ne pourrai considérer des échanges privés sur messageries.⁶

Vous êtes ainsi invités à interagir lors de réunions, via votre canal *Mattermost* ou via *Git*.⁷ Ce n'est pas obligatoire, mais en cas de tension au sein du groupe, ou si j'ai un doute sur le déroulé du projet, je ne prendrai en compte que les échanges sur ces plateformes ou transcrits dans des comptes-rendus de réunions.

Je demande à chacun vous de s'inscrire dans 4 canaux Mattermost : canal (privé) de votre groupe, le canal public d'annonce, le canal public de questions divers, et le canal public de votre rôle (j'ai créé un canal par rôle où vous pourrez interagir avec vos camarades d'autres équipes et me poser publiquement des questions relatives à ce rôle). Les questions posées en privé (par mail ou sur Mattermost) qui ne relèvent pas de votre vie privée et qui pourraient intéresser d'autres utilisateurs seront copiées (et anonymisées) sur un des canaux publics.

Tuteurs À chaque groupe sera assigné un tuteur. Ce sont des étudiants de master ou de l'école d'ingénieur (et si possible alumni de la licence). Vous aurez trois séances de tutorat avec votre tuteur. Vous pourrez communiquer avec lui via Mattermost : il sera invité sur votre canal privé (ainsi que moi) et il aura accès aux canaux publics. Le tuteur participera aussi aux soutenances, non pas pour évaluer, mais pour m'assister, servir de témoins, et vous aider lorsqu'il y a des incompréhensions. Je vous conseille vivement de profiter des séances de tutorat !

6. À moins de harcèlement ou d'insulte, qui seront transmis à la commission disciplinaire.

7. avec des commentaires ou des noms de commits

A Annexes

A.1 Coordinateur

C'est un rôle qui nécessite des capacités humaines, théorique, d'organisation et du recul. C'est aussi le rôle avec les tâches les plus diverses car il doit comprendre ce que chacun fait. Par contre, la technique impliquée est moindre. Il lui faudra mettre en place tout ce qui permettra aux membres du groupe de travailler en synergie.

Il lui faudra travailler avec *Git*, *Mattermost*, ainsi que tout autre outil qu'il vous semblera judicieux d'utiliser. En particulier il devra créer un groupe privé *Mattermost* pour interagir avec les enseignants⁸ et un dépôt *Git* pour le rendu du projet. Ce dernier sera utilisé tout au long du semestre et son historique sera consulté pendant la soutenance.

Le coordinateur doit écrire les diagrammes de classe, de Gantt, etc. Il pourra aussi écrire des interfaces *Java* et des implémentations minimales afin de permettre aux différents membres du projet de travailler sans attendre les autres. Le coordinateur doit comprendre le code abstrait (méthodes publiques et structures choisies) de chacun afin de pouvoir aider à l'intégration des codes entre eux. Il lui faudra écrire quelques tests et les utiliser régulièrement pour repérer les bugs au plus tôt.

Enfin, le coordinateur doit organiser les réunions et récupérer (pas forcément écrire) des comptes-rendus de celles-ci. Il faut essayer d'avoir des réunions bi-mensuelles, quitte à être virtuelles et/ou partielles. S'il n'a pas à vocation imposer les deadlines, il doit vérifier que chacun en a et qu'elles sont cohérentes entre elles. Dans l'idéal, il serait bien que chacun se fixe des objectifs à faire d'une réunion à l'autre, quitte à ce que ce ne soit que de petites choses. Ça permet de garder une dynamique.

Attention, être coordinateur ne signifie pas être le chef, au contraire : le coordinateur doit essayer de répondre aux exigences de chacun. Il est aussi invité à participer, de manière secondaire, à chacun des codes des autres membres pour avoir une vision globale de l'état du projet.

Remarque : Si vous pensiez avoir moins à lire en étant coordinateur, détrompez-vous, il vous faut maintenant lire le rôle de chacun des membres de votre groupe pour pouvoir les accompagner ;-)

Évaluation du Coordinateur

Le coordinateur sera évalué sur les critères suivants :

- la cohésion et la cohérence dans le groupe, ainsi que les moyens mis en œuvre pour faire face aux difficultés humaines,
- la mise en place et la promotion de l'utilisation de *git* (tout le monde doit être capable de l'utiliser d'ici le rendu final),
- la gestion du temps,

8. vous faites ce que vous voulez entre étudiants mais il vous est conseillé d'interagir aussi via *mattermost* lorsque vous voulez laisser une trace

- la documentation (commentaires, README, makefile...),
- l'organisation des tests (unitaire et manuels), des débuggage, et des reviews,
- la vision d'ensemble, de ce qui a été fait et ce qu'il a encore à faire,
- ce que vous aurez programmé ici et là,
- la gestion spécifique efficace des capacités de chacun.

Responsable Interface Graphique (IG)

C'est un rôle qui nécessite un peu d'aisance dans l'utilisation des objets et une capacité à fouiller dans la documentation des bibliothèques de Swing (ou JavaFX) pour trouver comment faire chaque petite chose.

Il s'agit de faire l'affichage graphique du modèle qui, lui, sera géré par un de vos camarades. La grosse difficulté est d'arriver à prendre en main une technologie que vous ne connaissez pas sans aide. La tâche vous semblera insurmontable, mais il faut simplement se lancer, faire quelque chose, et accepter de tout refaire une fois que l'on a compris comment ça marche. Dites-vous bien qu'un développeur est confronté à cet exercice de manière très régulière.

Interactions du responsable IG

Le responsable IG doit connaître les choix faits pour le modèle, voire discuter avec le responsable en question afin de disposer des attributs et des méthodes dont il a besoin dans les classes qu'il doit afficher. Il doit discuter régulièrement avec le responsable Modèle afin d'afficher correctement le plateau de jeu dans son état courant. Lorsqu'il y aura un responsable Plateau, le responsable IG devra également dialoguer avec lui pour l'affichage de création ou de modification du plateau.

Évaluation du responsable IG

En plus de la présence des éléments ci-dessus, le responsable graphique sera évalué sur :

- la maîtrise de l'outil graphique choisi,
- l'utilisation des contrôleur (listeneurs...),
- la séparation MVC,
- la gestion des interruptions renvoyées par le code de vos camarades (le programme ne doit pas cracher ni bugger silencieusement),
- l'esthétique.

Responsable du modèle

C'est un rôle qui nécessite une aisance dans la création de classes avec héritage, mais aussi de l'algorithmique pour les parties avancées. C'est un rôle central, car il est à la charnière entre la partie graphique et la partie Persistance, il recevra donc des demandes contradictoires de l'un et de l'autre et devra trouver un moyen de contenter tout le monde. C'est aussi un rôle doit avancé rapidement.

Le but de ce responsable est de créer les classes qui représentent les composants du jeu. Par contre, il ne s'occupera de l'affichage proprement dit. Dans un second temps, c'est aussi lui qui calculera l'endroit où doivent être placées les boites et le personnage lors des déplacements. Les classes devront contenir les informations utiles à la partie graphique et à la partie Persistance sans que l'un ait à se soucier des attributs de l'autre.

Le responsable du modèle pourra prendre en charge les fonctionnalités du rôle Chemins si ce rôle n'est pas affecté.

Interactions du responsable “Modèle”

Le responsable du modèle doit répondre aux demandes de chacun et est donc le centre de l'attention, mais son propre travail ne dépend pas de celui des autres. Il n'y a que pour les parties “Persistance”, “Chemin” et “Plateau” qu'il devra travailler directement avec quelqu'un. Dans la première, il va falloir trouver un moyen d'enregistrer et de charger les plateaux de jeu. Dans la seconde, on ajoutera d'autres éléments “graphiques” importants au modèle (exposition, zoom, etc...) qui n'ont pas d'intérêt pour la partie métier. Vous serez invité à séparer chaque classe de votre modèle en deux classes (l'une contenant l'autre comme attribut), mais ce n'est pas une obligation.

Évaluation du responsable “Modèle”

En plus de la présence des éléments ci-dessus, le responsable du modèle sera évalué sur :

- la vitesse l'implémentation de la première partie,
- le choix des hiérarchies de classes et des interfaces java,
- la facilité d'utilisation pour les autres membres du groupe,
- la compréhension des algorithmes implémentés.

Responsable de la persistance

Il s'agit d'un rôle demandant de manipuler des fichiers textes et formatés en Java. Vous devrez respecter les formats spécifiés pour définir un plateau, stocker des actions ou appliquer/enregistrer une solution (le chemin défini par le joueur par exemple).

La notion de persistance, en informatique, concerne tout ce qui reste disponible lorsque l'on arrête et relance le programme. Cela inclut les fichiers de configuration, les bases de données, les sauvegardes, l'importation de bibliothèque, de modes, etc... Dans notre cas, il s'agit de pouvoir faire la sauvegarde d'une partie en cours et la restauration d'une partie, mais aussi la sauvegarde et la restauration des chemins.

Interactions du responsable “Persistance”

Dans un premier temps, la difficulté sera principalement d'insérer votre travail pour qu'il soit cohérent à la fois avec la partie “Modèle”.

Évaluation du responsable “Persistance”

En plus de la présence des éléments ci-dessus, le responsable “Persistance” sera évalué sur :

- Le respect des formats,
- La qualité de la documentation,
- L'utilisation de bibliothèques de manipulation de fichiers formatés.

Responsable de l’installation et du déploiement

C’est un rôle qui demande des compétences aussi bien en programmation qu’en système. Le responsable de cette tâche doit s’assurer que le logiciel produit par le groupe soit installable et déployable dans un environnement a priori inconnu. Pour cela, il se doit utiliser des outils tels que Ant, Maven ou Graddle. Il concevra les scripts d’installation et de déploiement.

Interactions du responsable “Installation et Déploiement”

Le responsable “Installation et déploiement” devra solliciter les responsables des différentes tâches et notamment les responsables “Interface Graphique”, “Modèle” et “Persistance” pour s’assurer que les bibliothèques nécessaires au fonctionnement du jeu soient identifiées et disponibles sur la machine sur laquelle le logiciel est installé.

Évaluation du responsable “Installation et Déploiement”

En plus de la présence des éléments ci-dessus, le responsable Installation et Déploiement sera évalué sur :

- les choix de logiciel d’installation,
- la facilité d’installation et notamment la prise en compte de différents cas de figure,
- la documentation du processus d’installation et de déploiement,
- (bonus) la fourniture d’une image docker.

Responsable des chemins

C'est un rôle qui nécessite de la rigueur et de l'aisance en code. Il s'agit d'implémenter des algorithmes pour trouver le plus court chemin du personnage vers le point cliqué (sans bouger de boîte). Il ne faut donc pas se tromper si on veut un fonctionnement sans bug.

Interactions du responsable “Chemins”

Le responsable “Chemins” aura essentiellement besoin de discuter avec le responsable du “Modèle”, des attributs et des méthodes dont il a besoin dans les différentes classes.

Évaluation du responsable “Chemins”

En plus de la présence des éléments ci-dessus, le responsable Chemins sera évalué sur :

- la rapidité de code d'une première version,
- le choix et le respect des algorithmes implémentés.

Responsable de l'édition de plateau

Il s'agit d'un rôle qui nécessite un aisance dans l'utilisation du modèle. Il demande aussi des qualités de conception et d'implémentation afin de permettre une édition simple de plateau que ce soit pour en créer ou en modifier.

Interactions du responsable “Édition de plateau”

Le responsable de l'édition de plateau interagira naturellement avec le responsable du “Modèle”, et le responsable de l’“Interface graphique”. Il pourra également être amené à dialoguer avec le responsable Persistance.

Évaluation du responsable “Édition de plateau”

En plus de la présence des éléments ci-dessus, le responsable “Édition de plateau” sera évalué sur :

- la qualité du code,
- l'esthétique de la solution proposée,
- la facilité d'édition d'un plateau,
- l'intégration de la fonctionnalité dans le logiciel final.

Responsable de la Résolution Automatique

C'est un rôle à la fois technique et de conception. Il doit être capable de trouver l'ensemble des déplacements permettant de résoudre un plateau sans action de l'utilisateur, tout en respectant les règles du jeu. Dans un premier, le responsable Résolution Automatique pourrait s'intéresser au Sokoban classique, puis, dans un deuxième temps au Sokoban récursif.

Interactions du responsable “Résolution Automatique”

Le responsable Résolution Automatique interagira avec le responsable “Modèle” et le responsable “Chemins”, afin d'utiliser correctement les composants développés dans ces deux tâches.

Évaluation du responsable “Résolution Automatique”

En plus de la présence des éléments ci-dessus, le responsable Résolution Automatique sera évalué sur :

- la pertinence des stratégies de résolution proposées,
- la qualité de l'implémentation des stratégies de résolution proposées,
- la réutilisation des composants du modèle et de la tâche Chemins.

B Annexe : recherche de chemin

Les algorithmes ci-dessous ne sont pas soumis à obligation : Vous pouvez en utiliser un autre, mais pas utiliser une bibliothèque spécialisée.

Remarque : Vous reverrez ces algorithmes (et plein d'autres) au N5 dans le cours d'algorithmique des graphes.

B.1 Parcours en largeur sur une matrice non orientée

Entrées :

- matrice M de taille $n \times n$ de booléens (vrai si case libre, faux sinon).
- coordonnées (x_1, y_1) de départ
- coordonnées (x_2, y_2) d'arrivée

Sortie : Vide si chemin non trouvé, chemin entre les coordonnées de départ et d'arrivée sinon.

Structures :

- Un chemin est une pile de coordonnée.
- On initialise F une file de chemins parcourus par l'algorithme
- On initialise $\mathcal{V}$ un ensemble de coordonnées visitées

Algorithme :

- Faire $F = \{[(x_1, y_1)]\}$ et $\mathcal{V} = \emptyset$
- Tant que F non vide, faire
 1. défiler (*deque*) c dans F ,
 2. regarder (*peak*) la tête (x, y) de c ,
 3. si $(x, y) = (x_2, y_2)$, renvoyer c
 4. si $x < 0$ ou $x \geq n$ ou $y < 0$ ou $y \geq n$, continuer,
 5. si $M[x, y]$ non libre, continuer,
 6. si $(x, y) \in \mathcal{V}$, continuer,
 7. ajouter (x, y) dans $\mathcal{V}$,
 8. faire quatre copies de c ,
 9. empiler (*push*), $(x + 1, y)$, $(x - 1, y)$, $(x, y + 1)$ ou $(x, y - 1)$ dans chacune des copies
 10. enfiler (*enque*) les quatre copies dans F
- renvoyer la pile vide.

Conseils (non obligatoires) d'implémentation :

- implémentation par liste chaînée des piles afin d'avoir la copie gratuite,
- implémentation par tableau de la file et de l'ensemble car ils sont bornés par $4 * n^2$ et n^2 pour n pas très grand.

B.2 Parcours en largeur sur un graphe d'arité 4 orienté

Entrées :

- Ensemble fini de positions P possibles.
- Proposition **estLibre** : $P \times \text{Dir}$ où **Dir** représente les 4 points cardinaux, et **estLibre**(p, d) signifie que p peut se déplacer dans la direction d sans bouger de caisse.
- Fonction **suivant** : $P \times \text{Dir} \rightarrow P$ qui donne la position suivante dans la direction donnée si elle est libre.
- Univers vu comme un dictionnaire $U : \text{Pos} \rightarrow \text{boolean}$.
- Coordonnées $p_1 \in P$ de départ.
- Coordonnées $p_2 \in P$ d'arrivée.

Sortie : Vide si chemin non trouvé, chemin entre les coordonnées de départ et d'arrivée sinon.

Structures :

- Un chemin est une pile de coordonnées.
- On initialise F une file de chemins parcourus par l'algorithme
- On initialise $\mathcal{V}$ un ensemble de coordonnées visitées

Algorithme :

- Faire $F = \{[p_1]\}$ et $\mathcal{V} = \{p_2\}$
- Tant que F non vide, faire
 1. défiler (*deque*) c dans F ,
 2. regarder (*peak*) la tête p de c ,
 3. si $d = d_2$, renvoyer p ,
 4. pour chacune des directions $d \in \text{Dir}$ telles que **estLibre**(p, d) :
 - (a) soit $p' = \text{suivant}(p, d)$,
 - (b) si $p' \in \mathcal{V}$ continuer,
 - (c) ajouter p' dans P ,
 - (d) faire une copie c' de c ,
 - (e) empiler (*push*), p' dans c'
 - (f) enfiler (*enque*) c' dans F
- renvoyer la pile vide.

Conseils (non obligatoires) d'implémentation :

- implémentation par liste chaînée des piles afin d'avoir la copie gratuite,
- implémentation par tableau de la file et de l'ensemble car ils sont bornés par $4 * n^2$ et n^2 pour n pas très grand.